

Opis problemu

Zadanie składało się z dwóch głównych części. Pierwszą z nich było znalezienie optymalnej ścieżki (pojedynczego przejazdu) pomiędzy przystankiem początkowym A a przystankiem końcowym B. Optymalizacja polegała na minimalizacji jednego z dwóch kryteriów (wybranych przez użytkownika): całkowitego czasu przejazdu lub liczby przesiadek.

Drugą częścią było rozwiązanie wariantu problemu komiwojażera (TSP), w którym należało znaleźć optymalną ścieżkę rozpoczynającą się w wierzchołku A, odwiedzającą wszystkie przystanki z zadanej listy L i wracającą do wierzchołka startowego A. Kryteria optymalizacji pozostawały takie same jak w części pierwszej. Ze względu na zależność kosztów przejścia od czasu (godziny odjazdu pociągów), do rozwiązania tego problemu należało użyć metaheurystyki.

Zaczynamy od importów i sparsowania plików z danymi oraz załadowania ich do Dataframe'ów z biblioteki Pandas. Usuwamy pliki **agency.csv** oraz **feed_info.csv**, jako że nie zawierają potrzebnych informacji

```
In [1]: import os
import pandas as pd
from pathlib import Path
from dataclasses import dataclass
from datetime import datetime, date, timedelta
import random
from data_loader import create_transfer_edges, load_calendar, load_calendar_date
from utils import reconstruct_path, format_time, get_stop_name, _time_to_seconds
from cost_functions import h_cost_time, h_cost_transfers, g_cost_time, g_cost_tr
import heapq
import time
from collections import deque

data_path = os.path.join(os.getcwd(), "google_transit")
entries = os.listdir(data_path)
entries = [x for x in entries if x not in ["agency.csv", "feed_info.csv"]]
dfs = {}
for entry in entries:
    dfs[entry] = (pd.read_csv(os.path.join(data_path, entry)))
```

Reprezentacja danych (GTFS → graf)

Dane w formacie GTFS zostały przekształcone w skierowany graf zależny od czasu.

- **Wierzchołki:** Reprezentują poszczególne perony i stacje (na podstawie pliku `stops.txt`).
- **Krawędzie regularne:** Reprezentują bezpośrednie połączenia między kolejnymi przystankami w ramach danego kursu pociągu (plik `stop_times.txt`).

- **Krawędzie przesiadkowe:** Modelują piesze przejścia pomiędzy peronami należącymi do tej samej stacji nadrzędnej (`parent_station`). Przejście to dodaje stały czas oczekiwania.
- **Kalendarz kursowania:** Do określenia, czy pociąg faktycznie kursuje danego dnia, zaimplementowano sprawdzanie tygodniowych wzorców (`calendar.txt`) oraz wyjątków świątecznych (`calendar_dates.txt`).
- **Obiekt trasy:** służy do odtwarzania trasy po wykonaniu algorytmu

Największym problemem w reprezentacji był tzw. *day offset*. W formacie GTFS czasy mogą przekraczać 24 godziny (np. 25:10:00 dla kursu po północy). Aby algorytm płynnie radził sobie z przesiadkami nocnymi i wielodniowymi podróżami, czas w grafie jest śledzony jako absolutna liczba sekund od północy dnia startowego. Krawędzie są dynamicznie rzutowane na odpowiednie "okna dniowe" (wczoraj, dzisiaj, jutro), co powinno wyeliminować większość luk w rozkładach.

```
In [2]: @dataclass
class BaseEdge:
    target_stop_id: str
    is_transfer: bool

@dataclass
class TransferEdge(BaseEdge):
    transfer_time: int = 60

@dataclass
class RegularEdge(BaseEdge):
    target_stop_id: str
    departure_time: int
    arrival_time: int
    route_name: str
    service_id: str
    date_start: str = ""
    date_end: str = ""
    is_transfer = False

@dataclass
class Node:
    stop_id: str
    stop_name: str
    stop_lat: float
    stop_lon: float
    type: int
    parent_station: str

@dataclass
class RouteObject:
    origin_node_id: str
    edge_used: BaseEdge
    is_transfer: bool
```

```
In [3]: class TransitCalendar:
    def __init__(self):
        self.regular_schedules = {}
        self.exceptions = {}
    def set_schedules(self, schedules: dict):
```

```

        self.regular_schedules = schedules
    def set_exceptions(self, exceptions: dict):
        self.exceptions = exceptions

    def is_active(self, service_id: str, date_str: str) -> bool:
        if service_id in self.exceptions:
            if date_str in self.exceptions[service_id]['removed']:
                return False
            if date_str in self.exceptions[service_id]['added']:
                return True

        if service_id not in self.regular_schedules:
            return False

        schedule = self.regular_schedules[service_id]

        if not (schedule['start'] <= date_str <= schedule['end']):
            return False

        year = int(date_str[:4])
        month = int(date_str[4:6])
        day = int(date_str[6:8])
        weekday = date(year, month, day).weekday()
        return schedule['days'][weekday] == 1

```

In [4]:

```

def add_days_to_date(date_str: str, days: int) -> str:
    # date_str to "YYYYMMDD" helper dla TransitGraph
    dt = datetime.strptime(date_str, "%Y%m%d")
    dt += timedelta(days=days)
    return dt.strftime("%Y%m%d")

class TransitGraph():
    def __init__(self):
        self.nodes : dict[str, Node] = {}
        self.adjacent : dict[str, list[BaseEdge]] = {}
        self.calendar: TransitCalendar = {}
    def add_nodes(self, nodes: list[Node]):
        for entry in nodes:
            self.nodes[entry.stop_id]=entry
    def add_edge(self, source_node_id: str, edge: BaseEdge):
        curr = self.adjacent.get(source_node_id, [])
        curr.append(edge)
        self.adjacent[source_node_id] = curr
    def add_edges(self, edge_dict: dict[str, BaseEdge]):
        for key, value in edge_dict.items():
            curr = self.adjacent.get(key, [])
            curr += value
            self.adjacent[key] = curr
    def set_calendar(self, calendar: TransitCalendar):
        self.calendar= calendar

    def get_valid_neighbours(self, source_node_id: str, current_time: int, start
        valid_moves = []
        neighbours = self.adjacent.get(source_node_id, [])

        soonest_regular_edges = {}

        for edge in neighbours:

```

```

        if getattr(edge, 'is_transfer', False):
            arrival_time = current_time + edge.transfer_time
            valid_moves.append((edge, arrival_time))

        else:
            current_day_offset = current_time // 86400
            # dynamicznie sprawdzanie okien kursów
            for day_offset in [current_day_offset - 1, current_day_offset, c
                               check_date = add_days_to_date(start_date, day_offset)

            shifted_departure = edge.departure_time + (day_offset * 86400)
            shifted_arrival = edge.arrival_time + (day_offset * 86400)

            if shifted_departure >= current_time:
                if self.calendar.is_active(edge.service_id, check_date):
                    key = (edge.target_stop_id, edge.route_name)

                    if key not in soonest_regular_edges or shifted_depar
                        soonest_regular_edges[key] = (edge, shifted_depa

        for edge, shifted_dep, shifted_arr in soonest_regular_edges.values():
            valid_moves.append((edge, shifted_arr))

        valid_moves.sort(key=lambda x: x[1])
        return valid_moves

    def check_content(self):
        print("Nodes count", len(self.nodes.keys()))
        print("Edges count", sum([len(x) for x in self.adjacent.values()]))

```

Przetwarzamy utworzone wcześniej Dataframe'y, do postaci zdefiniowanych klas.
 Importujemy zbudowane klasy i helpery

```

In [5]: df_stops = dfs["stops.csv"]
df_routes = dfs["routes.csv"]
df_trips = dfs["trips.csv"]
df_stop_times = dfs["stop_times.csv"]
df_calendar = dfs["calendar.csv"]
df_calendar_dates = dfs["calendar_dates.csv"]

graph = TransitGraph()
nodes, grouped_nodes = load_stops(df_stops)
graph.add_nodes(nodes)

transfer_edges = create_transfer_edges(grouped_nodes, 60)
graph.add_edges(transfer_edges)
load_edges(df_routes, df_trips, df_stop_times, graph, df_calendar)

calendar = TransitCalendar()
calendar.set_exceptions(load_calendar_dates(df_calendar_dates))
calendar.set_schedules(load_calendar(df_calendar))
graph.set_calendar(calendar)

graph.check_content()

```


Created 1108 nodes
Created 2362 transfer edges
Created 45486 edges
Created 1123 exceptions
Created 3263 schedules
Nodes count 1108
Edges count 47847

Algorytmy

Dijkstra

Algorytm Dijkstry służy do wyszukiwania najkrótszej ścieżki w grafie z nieujemnymi wagami krawędzi. W naszej implementacji posłużył jako algorytm bazowy dla optymalizacji czasu przejazdu (wyszukiwanie wszecz bez heurystyki). Algorytm ten w każdym kroku wybiera z kolejki priorytetowej wierzchołek o najmniejszym znanym koszcie dotarcia od źródła. Został on zaimplementowany poprzez wywołanie głównej funkcji wyszukującej z heurystyką zwracającą zawsze wartość 0.

A^*

Algorytm A^* jest heurystycznym rozszerzeniem algorytmu Dijkstry, optymalizującym proces wyszukiwania poprzez kierowanie się w stronę celu. Minimalizuje on funkcję $f(curr, next) = g(curr, next) + h(next, end)$ gdzie h jest heurystyczną funkcją kosztu.

Mając to na uwadze, obydwa algorytmy jesteśmy w stanie zapisać za pomocą jednej funkcji, która jako argumenty przyjmuje funkcje kosztów.

- **Dla kryterium czasu:** Jako funkcję heurystyczną h zastosowano odległość geograficzną (liczoną ze wzoru Haversine'a na podstawie współrzędnych ze `stops.txt`) podzieloną przez maksymalną zakładaną prędkość pociągu.
- **Dla kryterium przesiadek:** Zastosowano podejście leksykograficzne. Koszt g obliczano ze wzoru $Cost = Transfers \cdot 10^{10} + Time$. Dzięki temu algorytm bezwzględnie minimalizuje liczbę przesiadek, a czas trwania podróży służy jako czynnik rozstrzygający remisy.

Implementacja algorytmu

```
In [6]: def find_path(start_id: str, end_id: str, start_time: int, start_date: str, graph,
open_set = []
start_state = (start_id, "transfer")

start_node = graph.nodes.get(start_id)
end_node = graph.nodes.get(end_id)
if not start_node or not end_node:
    return None, float('inf')

start_g = g_cost_func(current_g=0, is_transfer=False, arrival_time=start_time)
h_start = h_cost_func(start_node, end_node)
```

```

g_scores = {start_state: start_g}
came_from = {}

heapq.heappush(open_set, (start_g + h_start, start_g, start_time, start_id,
closed_set = set()

while open_set:
    curr_f, curr_g, curr_time, curr_id, curr_route = heapq.heappop(open_set)
    curr_state = (curr_id, curr_route)

    curr_node = graph.nodes[curr_id]
    curr_group = curr_node.parent_station if curr_node.parent_station else c
    end_group = end_node.parent_station if end_node.parent_station else end_

    is_goal = (curr_group == end_group)

    if is_goal:
        return reconstruct_path(came_from, curr_state), curr_g

    if curr_state in closed_set:
        continue

    closed_set.add(curr_state)
    neighbours = graph.get_valid_neighbours(curr_id, curr_time, start_date)

    for edge, arrival_time in neighbours:
        next_id = edge.target_stop_id
        is_transfer = isinstance(edge, TransferEdge)
        next_route = "transfer" if is_transfer else getattr(edge, 'route_name')
        next_state = (next_id, next_route)

        if next_state in closed_set:
            continue

        is_actual_transfer = is_transfer or (curr_route != "transfer" and cu

        tentative_g = g_cost_func(
            current_g=curr_g,
            is_transfer=is_actual_transfer,
            arrival_time=arrival_time,
            is_start=False
        )

        if next_state not in g_scores or tentative_g < g_scores[next_state]:
            came_from[next_state] = {
                'prev_state': curr_state,
                'edge_used': edge,
                'arrival_time': arrival_time
            }
            g_scores[next_state] = tentative_g

            next_node_obj = graph.nodes[next_id]
            h_cost = h_cost_func(next_node_obj, end_node)
            f_cost = tentative_g + h_cost

            heapq.heappush(open_set, (f_cost, tentative_g, arrival_time, nex

return None, float('inf')

```

Funkcja do rekonstrukcji ścieżki zwraca listę RouteObjects

```
In [7]: def reconstruct_path(came_from: dict, current_state: tuple):
    path = []

    while current_state in came_from:
        entry = came_from[current_state]

        prev_state = entry['prev_state']
        edge = entry['edge_used']

        is_transfer = isinstance(edge, TransferEdge)

        prev_node_id = prev_state[0]

        routeob = RouteObject(
            origin_node_id=prev_node_id,
            edge_used=edge,
            is_transfer=is_transfer
        )

        path.append(routeob)

        current_state = prev_state

    path.reverse()
    return path
```

Testow algorytmu dla kilku przypadków

```
In [8]: def test_algorithms(start_id, end_id, start_time, start_date, graph, dfs):
    stops_df = dfs["stops.csv"]

    algorithms = [
        ("Dijkstra", g_cost_time, lambda node, target: 0),
        ("A* czas", g_cost_time, h_cost_time),
        ("A* przesiadki", g_cost_transfers, h_cost_transfers)
    ]

    for name, g_func, h_func in algorithms:
        print(f"\n=== {name} ===")
        alg_start_time = time.perf_counter()
        path, cost = find_path(start_id, end_id, start_time, start_date, graph,
                               alg_end_time = time.perf_counter()
                               duration = alg_end_time - alg_start_time

        if not path:
            print("Brak ścieżki")
            continue

        visited_nodes = {start_id}
        start_time_real = None
        end_time_real = None
        prev_time = start_time
        day_offset = 0

        for entry in path:
            if hasattr(entry.edge_used, 'departure_time') and start_time_real is
```

```

        start_time_real = entry.edge_used.departure_time

        if hasattr(entry.edge_used, 'arrival_time'):
            cur_arrival = entry.edge_used.arrival_time
            if end_time_real is not None and cur_arrival < end_time_real:
                day_offset += 1
            end_time_real = cur_arrival

        visited_nodes.add(entry.edge_used.target_stop_id)

    if start_time_real is None or end_time_real is None:
        print("Błąd w obliczeniach czasu")
        continue

    start_dt = datetime.strptime(start_date, "%Y%m%d") + timedelta(seconds=s)
    end_dt = datetime.strptime(start_date, "%Y%m%d") + timedelta(seconds=end)

    transfer_count = 0
    prev_train_route = None

    for entry in path[:-1]:
        if entry.edge_used.is_transfer:
            current_route = "transfer"
        else:
            current_route = entry.edge_used.route_name
        if prev_train_route is not None and current_route != prev_train_route:
            transfer_count += 1

        prev_train_route = current_route

    print(f"Start: {stops_df.loc[stops_df['stop_id'].astype(str)==str(start_id)]}")
    print(f"Koniec: {stops_df.loc[stops_df['stop_id'].astype(str)==str(end_id)]}")
    print(f>Data startu: {start_dt.date()} {start_dt.time().strftime('%H:%M')}")
    print(f>Data zakończenia: {end_dt.date()} {end_dt.time().strftime('%H:%M')}")
    print(f"Liczba przesiadek: {transfer_count}")
    print(f"Liczba odwiedzonych przystanków: {len(visited_nodes)}")
    print(f"Czas przeszukiwania trasy: {duration:.4f}s")

```

```

In [9]: def run_test_suite(graph, dfs):
    test_cases = [
        ("1413064", "1413065", 10*3600, "20260306"),
        ("1413065", "1413066", 10*3600, "20260306"),
        ("1413066", "1413067", 10*3600, "20260306"),
        ("1413064", "1413067", 9*3600 + 30*60, "20260306"),
        ("1413064", "1413066", 8*3600 + 45*60, "20260306")
    ]

    for idx, (start_id, end_id, start_time, start_date) in enumerate(test_cases):
        start_name = get_stop_name(dfs["stops.csv"], start_id)
        end_name = get_stop_name(dfs["stops.csv"], end_id)
        print("\n" + "#" * 60)
        print(f"TEST {idx}: {start_name} ({start_id}) -> {end_name} ({end_id})")
        print(f>Data: {start_date}, start godzina: {format_time(start_time)}")
        print("#" * 60)

        test_algorithms(start_id, end_id, start_time, start_date, graph, dfs)

```

```
run_test_suite(graph, dfs)
```

```
#####  
TEST 1: Arnsdorf (b Dresden) (1413064) -> Bardo Przyłęk (1413065)  
Data: 20260306, start godzina: 10:00:00  
#####
```

```
=== Dijkstra ===  
Start: Arnsdorf (b Dresden) (1413064)  
Koniec: Bardo Przyłęk (1413065)  
Data startu: 2026-03-06 10:53  
Data zakończenia: 2026-03-06 15:34  
Liczba przesiadek: 2  
Liczba odwiedzonych przystanków: 32  
Czas przeszukiwania trasy: 2.3836s
```

```
=== A* czas ===  
Start: Arnsdorf (b Dresden) (1413064)  
Koniec: Bardo Przyłęk (1413065)  
Data startu: 2026-03-06 10:53  
Data zakończenia: 2026-03-06 15:34  
Liczba przesiadek: 2  
Liczba odwiedzonych przystanków: 32  
Czas przeszukiwania trasy: 1.8374s
```

```
=== A* przesiadki ===  
Start: Arnsdorf (b Dresden) (1413064)  
Koniec: Bardo Przyłęk (1413065)  
Data startu: 2026-03-06 10:53  
Data zakończenia: 2026-03-07 11:25  
Liczba przesiadek: 1  
Liczba odwiedzonych przystanków: 47  
Czas przeszukiwania trasy: 1.1734s
```

```
#####  
TEST 2: Bardo Przyłęk (1413065) -> Bardo Śląskie (1413066)  
Data: 20260306, start godzina: 10:00:00  
#####
```

```
=== Dijkstra ===  
Start: Bardo Przyłęk (1413065)  
Koniec: Bardo Śląskie (1413066)  
Data startu: 2026-03-06 11:25  
Data zakończenia: 2026-03-06 11:27  
Liczba przesiadek: 0  
Liczba odwiedzonych przystanków: 3  
Czas przeszukiwania trasy: 0.2364s
```

```
=== A* czas ===  
Start: Bardo Przyłęk (1413065)  
Koniec: Bardo Śląskie (1413066)  
Data startu: 2026-03-06 11:25  
Data zakończenia: 2026-03-06 11:27  
Liczba przesiadek: 0  
Liczba odwiedzonych przystanków: 3  
Czas przeszukiwania trasy: 0.1400s
```

```
=== A* przesiadki ===  
Start: Bardo Przyłęk (1413065)  
Koniec: Bardo Śląskie (1413066)  
Data startu: 2026-03-06 11:25  
Data zakończenia: 2026-03-06 11:27
```

Liczba przesiadek: 0
Liczba odwiedzonych przystanków: 3
Czas przeszukiwania trasy: 0.0654s

TEST 3: Bardo Śląskie (1413066) -> Bartnica (1413067)
Data: 20260306, start godzina: 10:00:00
#####

=== Dijkstra ===
Start: Bardo Śląskie (1413066)
Koniec: Bartnica (1413067)
Data startu: 2026-03-06 11:28
Data zakończenia: 2026-03-06 13:01
Liczba przesiadek: 1
Liczba odwiedzonych przystanków: 14
Czas przeszukiwania trasy: 2.5127s

=== A* czas ===
Start: Bardo Śląskie (1413066)
Koniec: Bartnica (1413067)
Data startu: 2026-03-06 11:28
Data zakończenia: 2026-03-06 13:01
Liczba przesiadek: 1
Liczba odwiedzonych przystanków: 14
Czas przeszukiwania trasy: 2.1491s

=== A* przesiadki ===
Start: Bardo Śląskie (1413066)
Koniec: Bartnica (1413067)
Data startu: 2026-03-06 11:28
Data zakończenia: 2026-03-06 13:01
Liczba przesiadek: 1
Liczba odwiedzonych przystanków: 15
Czas przeszukiwania trasy: 1.2785s

TEST 4: Arnsdorf (b Dresden) (1413064) -> Bartnica (1413067)
Data: 20260306, start godzina: 09:30:00
#####

=== Dijkstra ===
Start: Arnsdorf (b Dresden) (1413064)
Koniec: Bartnica (1413067)
Data startu: 2026-03-06 10:53
Data zakończenia: 2026-03-06 17:17
Liczba przesiadek: 3
Liczba odwiedzonych przystanków: 38
Czas przeszukiwania trasy: 3.9649s

=== A* czas ===
Start: Arnsdorf (b Dresden) (1413064)
Koniec: Bartnica (1413067)
Data startu: 2026-03-06 10:53
Data zakończenia: 2026-03-06 17:17
Liczba przesiadek: 3
Liczba odwiedzonych przystanków: 38
Czas przeszukiwania trasy: 3.7541s

=== A* przesiadki ===

Start: Arnsdorf (b Dresden) (1413064)
Koniec: Bartnica (1413067)
Data startu: 2026-03-06 10:53
Data zakończenia: 2026-03-07 06:50
Liczba przesiadek: 2
Liczba odwiedzonych przystanków: 32
Czas przeszukiwania trasy: 4.3213s

```
#####  
TEST 5: Arnsdorf (b Dresden) (1413064) -> Bardo Śląskie (1413066)  
Data: 20260306, start godzina: 08:45:00  
#####
```

=== Dijkstra ===
Start: Arnsdorf (b Dresden) (1413064)
Koniec: Bardo Śląskie (1413066)
Data startu: 2026-03-06 08:53
Data zakończenia: 2026-03-06 13:20
Liczba przesiadek: 2
Liczba odwiedzonych przystanków: 33
Czas przeszukiwania trasy: 1.9191s

=== A* czas ===
Start: Arnsdorf (b Dresden) (1413064)
Koniec: Bardo Śląskie (1413066)
Data startu: 2026-03-06 08:53
Data zakończenia: 2026-03-06 13:20
Liczba przesiadek: 2
Liczba odwiedzonych przystanków: 33
Czas przeszukiwania trasy: 1.3190s

=== A* przesiadki ===
Start: Arnsdorf (b Dresden) (1413064)
Koniec: Bardo Śląskie (1413066)
Data startu: 2026-03-06 08:53
Data zakończenia: 2026-03-07 11:27
Liczba przesiadek: 1
Liczba odwiedzonych przystanków: 48
Czas przeszukiwania trasy: 1.1758s

Funkcja dla zadania 1 podana w treści

```
In [10]: def run_pathfind(A: str, B: str, criteria: str, start_time_sec: int, start_date:
        """
        Funkcja obliczająca i wypisująca optymalną ścieżkę z A do B wg założeń zadan
        """
        t0 = time.perf_counter()

        path, path_cost = find_path(A, B, start_time_sec, start_date, graph, g_func,

        t1 = time.perf_counter()
        duration = t1 - t0

        if not path:
            print("Brak trasy spełniającej warunki.")
            return None, float('inf'), duration

        stops_df = dfs["stops.csv"]
        total_transfers = 0
        prev_train_route = None
```

```

for entry in path:
    is_transfer = getattr(entry, 'is_transfer', False) or getattr(entry.edge
    if is_transfer:
        continue

    origin_id = entry.orgin_node_id
    target_id = entry.edge_used.target_stop_id

    origin_name = get_stop_name(stops_df, origin_id)
    target_name = get_stop_name(stops_df, target_id)

    route_name = getattr(entry.edge_used, 'route_name', 'UNKNOWN')
    dep_time = format_time(entry.edge_used.departure_time)
    arr_time = format_time(entry.edge_used.arrival_time)

    print(f"{origin_name} -> {target_name} | linia: {route_name} | {dep_time

    if prev_train_route is not None and route_name != prev_train_route:
        total_transfers += 1
    prev_train_route = route_name

print()
transfer_count = 0
prev_train_route = None

for entry in path[:-1]:
    if entry.edge_used.is_transfer:
        current_route = "transfer"
    else:
        current_route = entry.edge_used.route_name
    if prev_train_route is not None and current_route != prev_train_route an
        transfer_count += 1

    prev_train_route = current_route

LARGE_VAL = 10**10
final_arrival_time = path_cost % LARGE_VAL if criteria == 'p' else path_cost

if criteria == 't':
    total_cost = (final_arrival_time - start_time_sec)
else:
    total_cost = (total_transfers * LARGE_VAL) + (final_arrival_time - start

delta = final_arrival_time - start_time_sec
hours = delta//3600
delta %= 3600
minutes = delta//60
delta %= 60
seconds = delta
print(f"Czas całej podróży {hours}h {minutes}m {seconds}s")
print(f"Liczba przesiadek {transfer_count}")

print(f"Koszt (min_kryterium): {total_cost}")
print(f"Czas obliczen [s]: {duration:.4f}\n")

return path, total_cost, duration

```

```
In [11]: run_pathfind("1413064", "1413065", "t", 10*3600, "20260306", graph, dfs, g_cost_t  
run_pathfind("1413064", "1413065", "p", 10*3600, "20260306", graph, dfs, g_cost_t
```

Arnsdorf (b Dresden) -> Bischofswerda | linia: D10 | 10:53:00 - 11:03:00
Bischofswerda -> Bautzen | linia: D10 | 11:07:00 - 11:19:00
Bautzen -> Löbau (Sachs) | linia: D10 | 11:19:00 - 11:33:00
Löbau (Sachs) -> Görlitz | linia: D10 | 11:33:00 - 11:48:00
Görlitz -> Zgorzelec | linia: D10 | 11:51:00 - 11:54:00
Zgorzelec -> Zgorzelec Miasto | linia: D10 | 12:19:00 - 12:21:00
Zgorzelec Miasto -> Pieńsk | linia: D10 | 12:21:00 - 12:27:00
Pieńsk -> Węgliniec | linia: D10 | 12:27:00 - 12:34:00
Węgliniec -> Zebrzydowa | linia: D10 | 12:35:00 - 12:42:00
Zebrzydowa -> Bolesławiec | linia: D10 | 12:42:00 - 12:49:00
Bolesławiec -> Chojnów | linia: D10 | 12:50:00 - 13:02:00
Chojnów -> Legnica | linia: D10 | 13:03:00 - 13:14:00
Legnica -> Środa Śląska | linia: D10 | 13:15:00 - 13:30:00
Środa Śląska -> Wrocław Nowy Dwór | linia: D10 | 13:30:00 - 13:43:00
Wrocław Nowy Dwór -> Wrocław Muchobór | linia: D10 | 13:44:00 - 13:46:00
Wrocław Muchobór -> Wrocław Główny | linia: D10 | 13:47:00 - 13:53:00
Wrocław Główny -> Iwiny | linia: D3/D9 | 14:28:00 - 14:33:00
Iwiny -> Smardzów Wrocławski | linia: D3/D9 | 14:34:00 - 14:36:00
Smardzów Wrocławski -> Żórawina | linia: D3/D9 | 14:36:00 - 14:40:00
Żórawina -> Węgry | linia: D3/D9 | 14:40:00 - 14:44:00
Węgry -> Boreczek | linia: D3/D9 | 14:45:00 - 14:48:00
Boreczek -> Warkocz | linia: D3/D9 | 14:49:00 - 14:53:00
Warkocz -> Strzelin | linia: D3/D9 | 14:53:00 - 14:57:00
Strzelin -> Biały Kościół | linia: D3/D9 | 14:58:00 - 15:02:00
Biały Kościół -> Henryków | linia: D3/D9 | 15:03:00 - 15:07:00
Henryków -> Ziębice | linia: D3/D9 | 15:08:00 - 15:12:00
Ziębice -> Starczów | linia: D3/D9 | 15:17:00 - 15:23:00
Starczów -> Kamieniec Ząbkowicki | linia: D3/D9 | 15:23:00 - 15:27:00
Kamieniec Ząbkowicki -> Bardo Przyłęk | linia: D3/D9 | 15:28:00 - 15:34:00

Czas całej podróży 5h 34m 0s

Liczba przesiadek 2

Koszt (min_kryterium): 20040

Czas obliczeń [s]: 1.8154

Arnsdorf (b Dresden) -> Bischofswerda | linia: D10 | 10:53:00 - 11:03:00
Bischofswerda -> Bautzen | linia: D10 | 11:07:00 - 11:19:00
Bautzen -> Löbau (Sachs) | linia: D10 | 11:19:00 - 11:33:00
Löbau (Sachs) -> Görlitz | linia: D10 | 11:33:00 - 11:48:00
Görlitz -> Zgorzelec | linia: D10 | 21:50:00 - 21:54:00
Zgorzelec -> Zgorzelec Miasto | linia: D10 | 21:54:00 - 21:56:00
Zgorzelec Miasto -> Jędrzychowice | linia: D10 | 21:57:00 - 21:59:00
Jędrzychowice -> Lasów | linia: D10 | 21:59:00 - 22:04:00
Lasów -> Pieńsk | linia: D10 | 22:04:00 - 22:06:00
Pieńsk -> Węgliniec | linia: D10 | 04:45:00 - 04:53:00
Węgliniec -> Zagajnik | linia: D10 | 04:54:00 - 04:58:00
Zagajnik -> Zebrzydowa | linia: D10 | 04:58:00 - 05:03:00
Zebrzydowa -> Bolesławiec | linia: D10 | 05:03:00 - 05:11:00
Bolesławiec -> Tomaszów Bolesławiecki | linia: D10 | 05:12:00 - 05:17:00
Tomaszów Bolesławiecki -> Okmiany | linia: D10 | 05:18:00 - 05:22:00
Okmiany -> Osetnica | linia: D10 | 05:23:00 - 05:26:00
Osetnica -> Chojnów | linia: D10 | 05:27:00 - 05:31:00
Chojnów -> Miłkowice | linia: D10 | 05:32:00 - 05:38:00
Miłkowice -> Legnica | linia: D10 | 05:38:00 - 05:46:00
Legnica -> Kunice | linia: D10 | 05:48:00 - 05:52:00
Kunice -> Jaśkowice Legnickie | linia: D10 | 05:53:00 - 05:56:00
Jaśkowice Legnickie -> Szczedrzykowice | linia: D10 | 05:56:00 - 05:59:00
Szchedrzykowice -> Malczyce | linia: D10 | 05:59:00 - 06:05:00
Malczyce -> Środa Śląska | linia: D10 | 06:05:00 - 06:11:00
Środa Śląska -> Przedmoście Święte | linia: D10 | 06:12:00 - 06:15:00

Przedmoście Święte -> Miękinia | linia: D10 | 06:16:00 - 06:19:00
Miękinia -> Mrozów | linia: D10 | 06:20:00 - 06:22:00
Mrozów -> Wrocław Leśnica | linia: D10 | 06:23:00 - 06:28:00
Wrocław Leśnica -> Wrocław Żerniki | linia: D10 | 06:29:00 - 06:32:00
Wrocław Żerniki -> Wrocław Nowy Dwór | linia: D10 | 06:32:00 - 06:35:00
Wrocław Nowy Dwór -> Wrocław Muchobór | linia: D10 | 06:36:00 - 06:38:00
Wrocław Muchobór -> Wrocław Główny | linia: D10 | 08:35:00 - 08:40:00
Wrocław Główny -> Iwiny | linia: D3/D9 | 10:21:00 - 10:26:00
Iwiny -> Smardzów Wrocławski | linia: D3/D9 | 10:27:00 - 10:29:00
Smardzów Wrocławski -> Żórawina | linia: D3/D9 | 10:30:00 - 10:34:00
Żórawina -> Węgry | linia: D3/D9 | 10:34:00 - 10:39:00
Węgry -> Boreczek | linia: D3/D9 | 10:40:00 - 10:44:00
Boreczek -> Warkocz | linia: D3/D9 | 10:44:00 - 10:48:00
Warkocz -> Strzelin | linia: D3/D9 | 10:48:00 - 10:52:00
Strzelin -> Biały Kościół | linia: D3/D9 | 10:53:00 - 10:57:00
Biały Kościół -> Henryków | linia: D3/D9 | 10:58:00 - 11:02:00
Henryków -> Ziębice | linia: D3/D9 | 11:03:00 - 11:07:00
Ziębice -> Starczów | linia: D3/D9 | 11:08:00 - 11:13:00
Starczów -> Kamieniec Ząbkowicki | linia: D3/D9 | 11:14:00 - 11:17:00
Kamieniec Ząbkowicki -> Bardo Przyłęk | linia: D3/D9 | 11:18:00 - 11:25:00

Czas całej podróży 25h 25m 0s

Liczba przesiadek 1

Koszt (min_kryterium): 10000091500

Czas obliczeń [s]: 1.3011

```
Out[11]: ([RouteObject(origin_node_id='1413064', edge_used=TransferEdge(target_stop_id='1474632', is_transfer=True, transfer_time=60), is_transfer=False),
RouteObject(origin_node_id='1474632', edge_used=RegularEdge(target_stop_id='1474629', is_transfer=False, departure_time=39180, arrival_time=39780, route_name='D10', service_id='152_397173', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1474629', edge_used=RegularEdge(target_stop_id='1474626', is_transfer=False, departure_time=40020, arrival_time=40740, route_name='D10', service_id='152_397173', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1474626', edge_used=RegularEdge(target_stop_id='1474622', is_transfer=False, departure_time=40740, arrival_time=41580, route_name='D10', service_id='152_397173', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1474622', edge_used=RegularEdge(target_stop_id='1474617', is_transfer=False, departure_time=41580, arrival_time=42480, route_name='D10', service_id='152_397173', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1474617', edge_used=RegularEdge(target_stop_id='1475133', is_transfer=False, departure_time=78600, arrival_time=78840, route_name='D10', service_id='126_397135', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1475133', edge_used=RegularEdge(target_stop_id='1475134', is_transfer=False, departure_time=78840, arrival_time=78960, route_name='D10', service_id='126_397135', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1475134', edge_used=RegularEdge(target_stop_id='1475187', is_transfer=False, departure_time=79020, arrival_time=79140, route_name='D10', service_id='126_397135', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1475187', edge_used=RegularEdge(target_stop_id='1475188', is_transfer=False, departure_time=79140, arrival_time=79440, route_name='D10', service_id='126_397135', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1475188', edge_used=RegularEdge(target_stop_id='1475135', is_transfer=False, departure_time=79440, arrival_time=79560, route_name='D10', service_id='126_397135', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1475135', edge_used=RegularEdge(target_stop_id='1475136', is_transfer=False, departure_time=17100, arrival_time=17580, route_name='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1475136', edge_used=RegularEdge(target_stop_id='1475140', is_transfer=False, departure_time=17640, arrival_time=17880, route_name='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1475140', edge_used=RegularEdge(target_stop_id='1475137', is_transfer=False, departure_time=17880, arrival_time=18180, route_name='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1475137', edge_used=RegularEdge(target_stop_id='1474945', is_transfer=False, departure_time=18180, arrival_time=18660, route_name='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1474945', edge_used=RegularEdge(target_stop_id='1475141', is_transfer=False, departure_time=18720, arrival_time=19020, route_name='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'), is_transfer=False),
RouteObject(origin_node_id='1475141', edge_used=RegularEdge(target_stop_id='1475142', is_transfer=False, departure_time=19080, arrival_time=19320, route_name
```

```
= 'D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475142', edge_used=RegularEdge(target_stop_id='14
75143', is_transfer=False, departure_time=19380, arrival_time=19560, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475143', edge_used=RegularEdge(target_stop_id='14
74946', is_transfer=False, departure_time=19620, arrival_time=19860, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1474946', edge_used=RegularEdge(target_stop_id='14
74845', is_transfer=False, departure_time=19920, arrival_time=20280, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1474845', edge_used=RegularEdge(target_stop_id='16
09726', is_transfer=False, departure_time=20280, arrival_time=20760, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1609726', edge_used=RegularEdge(target_stop_id='21
14837', is_transfer=False, departure_time=20880, arrival_time=21120, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='2114837', edge_used=RegularEdge(target_stop_id='14
75000', is_transfer=False, departure_time=21180, arrival_time=21360, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475000', edge_used=RegularEdge(target_stop_id='14
75001', is_transfer=False, departure_time=21360, arrival_time=21540, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475001', edge_used=RegularEdge(target_stop_id='14
75002', is_transfer=False, departure_time=21540, arrival_time=21900, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475002', edge_used=RegularEdge(target_stop_id='14
74949', is_transfer=False, departure_time=21900, arrival_time=22260, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1474949', edge_used=RegularEdge(target_stop_id='14
75003', is_transfer=False, departure_time=22320, arrival_time=22500, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475003', edge_used=RegularEdge(target_stop_id='14
75004', is_transfer=False, departure_time=22560, arrival_time=22740, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475004', edge_used=RegularEdge(target_stop_id='14
75005', is_transfer=False, departure_time=22800, arrival_time=22920, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475005', edge_used=RegularEdge(target_stop_id='14
75006', is_transfer=False, departure_time=22980, arrival_time=23280, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475006', edge_used=RegularEdge(target_stop_id='14
75007', is_transfer=False, departure_time=23340, arrival_time=23520, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1475007', edge_used=RegularEdge(target_stop_id='14
74950', is_transfer=False, departure_time=23520, arrival_time=23700, route_name
```

```
= 'D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1474950', edge_used=RegularEdge(target_stop_id='14
74793', is_transfer=False, departure_time=23760, arrival_time=23880, route_name
='D10', service_id='2304_399649', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1474793', edge_used=RegularEdge(target_stop_id='14
74640', is_transfer=False, departure_time=30900, arrival_time=31200, route_name
='D10', service_id='2310_399655', date_start='20260225', date_end='20260307'),
is_transfer=False),
    RouteObject(origin_node_id='1474640', edge_used=RegularEdge(target_stop_id='14
75191', is_transfer=False, departure_time=37260, arrival_time=37560, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1475191', edge_used=RegularEdge(target_stop_id='14
75192', is_transfer=False, departure_time=37620, arrival_time=37740, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1475192', edge_used=RegularEdge(target_stop_id='14
75193', is_transfer=False, departure_time=37800, arrival_time=38040, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1475193', edge_used=RegularEdge(target_stop_id='14
75194', is_transfer=False, departure_time=38040, arrival_time=38340, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1475194', edge_used=RegularEdge(target_stop_id='14
75195', is_transfer=False, departure_time=38400, arrival_time=38640, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1475195', edge_used=RegularEdge(target_stop_id='14
75196', is_transfer=False, departure_time=38640, arrival_time=38880, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1475196', edge_used=RegularEdge(target_stop_id='14
74641', is_transfer=False, departure_time=38880, arrival_time=39120, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1474641', edge_used=RegularEdge(target_stop_id='14
74853', is_transfer=False, departure_time=39180, arrival_time=39420, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1474853', edge_used=RegularEdge(target_stop_id='14
74852', is_transfer=False, departure_time=39480, arrival_time=39720, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1474852', edge_used=RegularEdge(target_stop_id='14
74642', is_transfer=False, departure_time=39780, arrival_time=40020, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1474642', edge_used=RegularEdge(target_stop_id='14
74851', is_transfer=False, departure_time=40080, arrival_time=40380, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1474851', edge_used=RegularEdge(target_stop_id='14
74643', is_transfer=False, departure_time=40440, arrival_time=40620, route_name
='D3/D9', service_id='2475_399834', date_start='20260225', date_end='2026030
7'), is_transfer=False),
    RouteObject(origin_node_id='1474643', edge_used=RegularEdge(target_stop_id='14
75207', is_transfer=False, departure_time=40680, arrival_time=41100, route_name
```



```
= 'D3/D9', service_id='2475_399834', date_start='20260225', date_end='20260307'), is_transfer=False)],  
10000091500,  
1.3011492000000544)
```

Tabu Search

Do rozwiązania problemu komiwojażera zastosowano metaheurystykę Tabu Search opartą na algorytmie Knoxa. Pozwala ona wyjść z maksimów lokalnych poprzez użycie listy Tabu zawierającej zakazane ruchy. Funkcja oceny wylicza całkowity czas podróży dla danej permutacji punktów do odwiedzenia poprzez, wywołanie algorytm A^* dla kolejnych par przystanków i aktualizując czas początkowy dla każdego następnego etapu.

Modyfikacje

W celu spełnienia wymagań i optymalizacji czasu działania, wprowadzono następujące modyfikacje do algorytmu Tabu Search:

- **Zmienny rozmiar listy Tabu (T):** Rozmiar kolejki jest dynamicznie dobierany na podstawie długości listy L do odwiedzenia ($t = \max(5, 1.5 \cdot |L|)$), zapobiegając szybkiemu zapętlaniu się dla małych sąsiedztw oraz oszczędzając pamięć dla dużych instancji.
- **Kryterium aspiracji:** Ruch znajdujący się na liście Tabu zostaje dopuszczony do wykonania, jeśli wygenerowane rozwiązanie jest lepsze od globalnie najlepszego dotychczas znalezionej rozwiązania s^* .
- **Próbkowanie sąsiedztwa:** Dla dłuższych list L pełne przeglądanie sąsiedztwa wymagałoby masy obliczeń, dlatego wprowadziłem ograniczenie i losowe próbkowanie badanych sąsiadów w iteracji.
- **Caching:** Modyfikacja drastycznie skracająca czas działania Tabu Search. Funkcja celu zapisuje wyniki zapytań do algorytmu A^* z informacją o dacie startu zapytania oraz właściwym czasie odjazdu odnalezionej pociągu. Kolejne wywołania dla tej samej pary stacji w iteracjach Tabu Search pomijają obliczenia, jeśli nowy czas przyjazdu na stację mieści się w "oknie oczekiwania" na znany już pociąg.

```
In [12]: class PathCacheEntry:  
    def __init__(self, start_time, departure_time, path, cost_or_inf):  
        self.start_time = start_time  
        self.departure_time = departure_time  
        self.path = path  
        self.cost_or_inf = cost_or_inf  
  
        self.initial_walk_time = 0  
        if path:  
            for e in path:  
                if getattr(e, 'is_transfer', False) or getattr(e, 'edge_used', 'is_'  
                    self.initial_walk_time += e.edge_used.transfer_time  
            else:  
                break
```

```

def is_valid_for(self, t):
    if not self.path:
        return False
    if self.departure_time is None:
        return t >= self.start_time
    return self.start_time <= t <= (self.departure_time - self.initial_walk_

def get_cost(self, t):
    if self.departure_time is None:
        return self.cost_or_inf - self.start_time + t
    return self.cost_or_inf

```

```

In [13]: class TabuSearchRouter:
    def __init__(self, start_id, L_str, criteria, start_time, start_date, graph,
                self.start_id = start_id
                if isinstance(L_str, str):
                    self.L_ids = [x.strip() for x in L_str.split(";") if x.strip()]
                else:
                    self.L_ids = list(L_str)
                self.criteria = criteria
                self.start_time_sec = _time_to_seconds(start_time)
                self.start_date = start_date
                self.graph = graph
                self.dfs = dfs
                self.g_func = g_func
                self.h_func = h_func

                self.cache = {}
                self.cache_hits = 0
                self.cache_misses = 0

    def _get_absolute_departure(self, start_time, path):
        walk_time = 0
        for e in path:
            if getattr(e, 'is_transfer', False) or getattr(e.edge_used, 'is_tran
                walk_time += e.edge_used.transfer_time
            else:
                gtfs_dep = e.edge_used.departure_time
                arrival_at_station = start_time + walk_time

                base_offset = arrival_at_station // 86400

                for day_offset in [base_offset - 1, base_offset, base_offset + 1
                    abs_dep = gtfs_dep + day_offset * 86400
                    if abs_dep >= arrival_at_station:
                        return abs_dep
                return gtfs_dep
        return None

    def _get_cached_path(self, origin, destination, current_time):
        entries = self.cache.get((origin, destination), [])
        for entry in entries:
            if entry.is_valid_for(current_time):
                self.cache_hits += 1
                return entry.path, entry.get_cost(current_time)
        self.cache_misses += 1
        return None, None

    def _cache_path(self, origin, destination, start_time, path, cost_or_inf):

```

```

key = (origin, destination)
if key not in self.cache:
    self.cache[key] = []

departure_time = None
if path:
    departure_time = self._get_absolute_departure(start_time, path)

if path and departure_time is not None:
    for entry in self.cache[key]:
        if entry.departure_time == departure_time:
            if start_time < entry.start_time:
                entry.start_time = start_time
            return

new_entry = PathCacheEntry(start_time, departure_time, path, cost_or_inf)
self.cache[key].append(new_entry)

def evaluate_permutation(self, perm):
    full_route = [self.start_id] + list(perm) + [self.start_id]

    current_time = self.start_time_sec

    total_path = []

    for i in range(len(full_route) - 1):
        origin = full_route[i]
        destination = full_route[i+1]

        leg_path, cost_or_inf = self._get_cached_path(origin, destination, c
        if leg_path is None:
            leg_path, cost_or_inf = find_path(origin, destination, current_t
            self._cache_path(origin, destination, current_time, leg_path, co

        if not leg_path or len(leg_path) == 0:
            return float('inf'), []

        total_path.extend(leg_path)

        LARGE_VAL = 10**10
        arrival_time = cost_or_inf % LARGE_VAL if self.criteria == 'p' else

        current_time = arrival_time

    total_transfers = 0
    prev_train_route = None

    for entry in total_path:
        if getattr(entry, 'is_transfer', False) or getattr(entry, 'edge_used',
            continue

        current_route = getattr(entry, 'edge_used', 'route_name', 'UNKNOWN')
        if prev_train_route is not None and current_route != prev_train_route:
            total_transfers += 1
        prev_train_route = current_route

    if self.criteria == 't':
        total_cost = current_time - self.start_time_sec
    else:
        LARGE_VAL = 10**10

```

```

        total_cost = (total_transfers * LARGE_VAL) + (current_time - self.start_time)

    return total_cost, total_path

def solve(self, max_iter=10, max_no_improve=5, max_sampled_neighbors=8):
    t0 = time.perf_counter()

    current_solution = list(self.L_ids)
    best_cost, best_full_path = self.evaluate_permutation(current_solution)
    best_solution = list(current_solution)

    tabu_tenure = max(5, int(1.5 * len(self.L_ids)))
    tabu_list = deque()
    tabu_set = set()

    def add_to_tabu(sol):
        tup = tuple(sol)
        if len(tabu_list) >= tabu_tenure:
            oldest = tabu_list.popleft()
            tabu_set.discard(oldest)
        tabu_list.append(tup)
        tabu_set.add(tup)

    add_to_tabu(current_solution)

    iterations_without_improvement = 0

    for iteration in range(max_iter):
        neighborhood = []
        for i in range(len(current_solution)):
            for j in range(i + 1, len(current_solution)):
                neighbor = list(current_solution)
                neighbor[i], neighbor[j] = neighbor[j], neighbor[i]
                neighborhood.append(neighbor)

        if len(neighborhood) > max_sampled_neighbors:
            neighborhood = random.sample(neighborhood, max_sampled_neighbors)

        best_neighbor = None
        best_neighbor_cost = float('inf')
        best_neighbor_path = []

        for neighbor in neighborhood:
            cost, path = self.evaluate_permutation(neighbor)
            is_tabu = tuple(neighbor) in tabu_set

            if is_tabu and cost < best_cost:
                is_tabu = False

            if not is_tabu and cost < best_neighbor_cost:
                best_neighbor_cost = cost
                best_neighbor = neighbor
                best_neighbor_path = path

        if best_neighbor is None:
            break

    current_solution = best_neighbor
    add_to_tabu(current_solution)

```

```

        if best_neighbor_cost < best_cost:
            best_cost = best_neighbor_cost
            best_solution = list(best_neighbor)
            best_full_path = best_neighbor_path
            iterations_without_improvement = 0
        else:
            iterations_without_improvement += 1

        if iterations_without_improvement >= max_no_improve:
            break

    t1 = time.perf_counter()
    duration = t1 - t0

    return best_solution, best_cost, best_full_path, duration

def print_solution(self, total_path, duration, total_cost):
    if not total_path:
        print(f"Czas obliczen [s]: {duration:.4f}")
        print("Brak ścieżki spełniającej warunki.")
        return

    stops_df = self.dfs["stops.csv"]

    for entry in total_path:
        is_transfer = getattr(entry, 'is_transfer', False) or getattr(entry,
        'is_transfer', False)
        if is_transfer:
            continue

        origin_id = entry.origin_node_id
        target_id = entry.edge_used.target_stop_id

        origin_name = get_stop_name(stops_df, origin_id)
        target_name = get_stop_name(stops_df, target_id)

        route_name = getattr(entry.edge_used, 'route_name', 'UNKNOWN')
        dep_time = format_time(entry.edge_used.departure_time)
        arr_time = format_time(entry.edge_used.arrival_time)

        print(f"{origin_name} -> {target_name} | linia: {route_name} | {dep_

    print()

    print(f"Koszt (min_kryterium): {total_cost}")
    print(f"Czas obliczen [s]: {duration:.4f}")
    print(f"Cache Statystyki: Hits={self.cache_hits}, Misses={self.cache_mis

```

Funkcja dla zadania 2 podana w treści zadania (tym razem wywołujemy po prostu metodę klasy)

```

In [14]: def run_tabu_search(A: str, L_str: str, criteria: str, start_time: str, start_da
router = TabuSearchRouter(A, L_str, criteria, start_time, start_date, graph,
best_solution, best_cost, best_full_path, duration = router.solve()
router.print_solution(best_full_path, duration, best_cost)
return best_solution, best_cost, best_full_path, duration

```

```

In [15]: A = '1413064'
L = ['1413065', '1413066', '1413067', '1413380']
criteria = 'time'

```

```

start_time = 10 * 3600
start_date = "20260306"

search_start = time.time()
best_solution, best_cost, best_full_path, duration = run_tabu_search(
    A=A,
    L_str=L,
    criteria='t',
    start_time=start_time,
    start_date=start_date,
    graph=graph,
    dfs = dfs,
    g_func=g_cost_time,
    h_func=h_cost_time,
)
search_duration = time.time() - search_start

# Display results
print(f"\nBEST SOLUTION FOUND:")
print(f"Route: {A} -> {' -> '}.join(best_solution) -> {A}")
print(f"Total cost: {best_cost:.2f}")
print(f"Search duration: {search_duration:.4f}s")

# Detailed path info if available
if best_full_path:
    print(f"\nDetailed path legs: {len(best_full_path)} segments")
else:
    print("\nNo detailed path information available")

```

Arnsdorf (b Dresden) -> Bischofswerda | linia: D10 | 10:53:00 - 11:03:00
Bischofswerda -> Bautzen | linia: D10 | 11:07:00 - 11:19:00
Bautzen -> Löbau (Sachs) | linia: D10 | 11:19:00 - 11:33:00
Löbau (Sachs) -> Görlitz | linia: D10 | 11:33:00 - 11:48:00
Görlitz -> Zgorzelec | linia: D10 | 11:51:00 - 11:54:00
Zgorzelec -> Zgorzelec Miasto | linia: D10 | 12:19:00 - 12:21:00
Zgorzelec Miasto -> Pieńsk | linia: D10 | 12:21:00 - 12:27:00
Pieńsk -> Węgliniec | linia: D10 | 12:27:00 - 12:34:00
Węgliniec -> Zembrzydowa | linia: D10 | 12:35:00 - 12:42:00
Zembrzydowa -> Bolesławiec | linia: D10 | 12:42:00 - 12:49:00
Bolesławiec -> Chojnów | linia: D10 | 12:50:00 - 13:02:00
Chojnów -> Legnica | linia: D10 | 13:03:00 - 13:14:00
Legnica -> Środa Śląska | linia: D10 | 13:15:00 - 13:30:00
Środa Śląska -> Wrocław Nowy Dwór | linia: D10 | 13:30:00 - 13:43:00
Wrocław Nowy Dwór -> Wrocław Muchobór | linia: D10 | 13:44:00 - 13:46:00
Wrocław Muchobór -> Wrocław Główny | linia: D10 | 13:47:00 - 13:53:00
Wrocław Główny -> Iwiny | linia: D3/D9 | 14:28:00 - 14:33:00
Iwiny -> Smardzów Wrocławski | linia: D3/D9 | 14:34:00 - 14:36:00
Smardzów Wrocławski -> Żórawina | linia: D3/D9 | 14:36:00 - 14:40:00
Żórawina -> Węgry | linia: D3/D9 | 14:40:00 - 14:44:00
Węgry -> Boreczek | linia: D3/D9 | 14:45:00 - 14:48:00
Boreczek -> Warkocz | linia: D3/D9 | 14:49:00 - 14:53:00
Warkocz -> Strzelin | linia: D3/D9 | 14:53:00 - 14:57:00
Strzelin -> Biały Kościół | linia: D3/D9 | 14:58:00 - 15:02:00
Biały Kościół -> Henryków | linia: D3/D9 | 15:03:00 - 15:07:00
Henryków -> Ziębice | linia: D3/D9 | 15:08:00 - 15:12:00
Ziębice -> Starczów | linia: D3/D9 | 15:17:00 - 15:23:00
Starczów -> Kamieniec Ząbkowicki | linia: D3/D9 | 15:23:00 - 15:27:00
Kamieniec Ząbkowicki -> Bardo Przyłęk | linia: D3/D9 | 15:28:00 - 15:34:00
Bardo Przyłęk -> Bardo Śląskie | linia: D3/D9 | 15:35:00 - 15:37:00
Bardo Śląskie -> Kłodzko Główne | linia: D91/D90 | 15:48:00 - 15:57:00
Kłodzko Główne -> Bierkowice | linia: D96 | 16:44:00 - 16:49:00
Bierkowice -> Gorzuchów Kłodzki | linia: D96 | 16:49:00 - 16:52:00
Gorzuchów Kłodzki -> Ścinawka Średnia | linia: D96 | 16:52:00 - 16:58:00
Ścinawka Średnia -> Nowa Ruda | linia: D96 | 16:58:00 - 17:06:00
Nowa Ruda -> Nowa Ruda Przedmieście | linia: D96 | 17:06:00 - 17:09:00
Nowa Ruda Przedmieście -> Nowa Ruda Zdrojowisko | linia: D96 | 17:10:00 - 17:17:00
0
Nowa Ruda Zdrojowisko -> Ludwikowice Kłodzkie | linia: D96 | 17:17:00 - 17:22:00
Ludwikowice Kłodzkie -> Świerki Dolne | linia: D96 | 17:22:00 - 17:27:00
Świerki Dolne -> Bartnica | linia: D96 | 17:27:00 - 17:29:00
Bartnica -> Głuszyca Górna | linia: D96 | 17:30:00 - 17:34:00
Głuszyca Górna -> Głuszyca | linia: D96 | 17:34:00 - 17:37:00
Głuszyca -> Jedlina-Zdrój | linia: D96 | 17:37:00 - 17:41:00
Jedlina-Zdrój -> Jedlina-Zdrój Centrum | linia: D64/D91 | 18:19:00 - 18:22:00
Jedlina-Zdrój Centrum -> Jugowice | linia: D64/D91 | 18:23:00 - 18:28:00
Jugowice -> Zagórze Śląskie | linia: D64/D91 | 18:29:00 - 18:32:00
Zagórze Śląskie -> Lubachów | linia: D64/D91 | 18:33:00 - 18:37:00
Lubachów -> Bystrzyca Górna | linia: D64/D91 | 18:37:00 - 18:40:00
Bystrzyca Górna -> Burkatów | linia: D64/D91 | 18:40:00 - 18:42:00
Burkatów -> Świdnica Miasto | linia: D64/D91 | 18:43:00 - 18:49:00
Świdnica Miasto -> Świdnica Zawiszów | linia: D64/D91 | 18:50:00 - 18:52:00
Świdnica Zawiszów -> Jaworzyna Śląska | linia: D64/D91 | 18:52:00 - 19:01:00
Jaworzyna Śląska -> Żarów | linia: D60 | 19:12:00 - 19:16:00
Żarów -> Imbramowice | linia: D60 | 19:17:00 - 19:22:00
Imbramowice -> Mietków | linia: D60 | 19:22:00 - 19:27:00
Mietków -> Kąty Wrocławskie | linia: D60 | 19:27:00 - 19:33:00
Kąty Wrocławskie -> Sadowice Wrocławskie | linia: D60 | 19:34:00 - 19:38:00
Sadowice Wrocławskie -> Smolec | linia: D60 | 19:39:00 - 19:43:00
Smolec -> Mokronos Górny | linia: D60 | 19:44:00 - 19:46:00

Mokronos Górny -> Wrocław Zachodni | linia: D60 | 19:47:00 - 19:50:00
Wrocław Zachodni -> Wrocław Grabiszyn | linia: D60 | 19:52:00 - 19:54:00
Wrocław Grabiszyn -> Wrocław Główny | linia: D60 | 19:56:00 - 20:03:00
Wrocław Główny -> Wrocław Muchobór | linia: D14 | 20:17:00 - 20:22:00
Wrocław Muchobór -> Wrocław Nowy Dwór | linia: D14 | 20:22:00 - 20:24:00
Wrocław Nowy Dwór -> Środa Śląska | linia: D14 | 20:25:00 - 20:40:00
Środa Śląska -> Legnica | linia: D14 | 20:41:00 - 21:00:00
Legnica -> Jezierzany | linia: D14 | 21:10:00 - 21:14:00
Jezierzany -> Miłkowice | linia: D14 | 21:15:00 - 21:19:00
Miłkowice -> Chojnów | linia: D1 | 21:41:00 - 21:47:00
Chojnów -> Osetnica | linia: D1 | 21:47:00 - 21:51:00
Osetnica -> Okmiany | linia: D1 | 21:52:00 - 21:55:00
Okmiany -> Tomaszów Bolesławiecki | linia: D1 | 21:56:00 - 22:00:00
Tomaszów Bolesławiecki -> Bolesławiec | linia: D1 | 22:01:00 - 22:08:00
Bolesławiec -> Zebrzydowa | linia: D10 | 23:16:00 - 23:23:00
Zebrzydowa -> Zagajnik | linia: D10 | 23:24:00 - 23:28:00
Zagajnik -> Węgliniec | linia: D10 | 23:28:00 - 23:33:00
Węgliniec -> Pieńsk | linia: D10 | 23:34:00 - 23:42:00
Pieńsk -> Jędrzychowice | linia: D10 | 23:42:00 - 23:47:00
Jędrzychowice -> Zgorzelec Miasto | linia: D10 | 23:48:00 - 23:50:00
Zgorzelec Miasto -> Zgorzelec | linia: D10 | 23:50:00 - 23:53:00
Zgorzelec -> Görlitz | linia: D10 | 05:15:00 - 05:19:00
Görlitz -> Löbau (Sachs) | linia: D10 | 08:13:00 - 08:28:00
Löbau (Sachs) -> Bautzen | linia: D10 | 08:28:00 - 08:42:00
Bautzen -> Bischofswerda | linia: D10 | 08:42:00 - 08:54:00
Bischofswerda -> Arnsdorf (b Dresden) | linia: D10 | 08:56:00 - 09:06:00

Koszt (min_kryterium): 83160
Czas obliczeń [s]: 79.4434
Cache Statystyki: Hits=102, Misses=53

BEST SOLUTION FOUND:

Route: 1413064 -> 1413065 -> 1413066 -> 1413067 -> 1413380 -> 1413064
Total cost: 83160.00
Search duration: 79.5957s

Detailed path legs: 97 segments

Eksperymenty i wyniki

Implementacja wykazuje stuprocentową poprawność przy zestawieniu z portalem pasażera.

Dla pojedynczych tras algorytm A^* ze zdefiniowaną heurystyką odległościową wyraźnie wygrywał czasowo z algorytmem Dijkstry. Z kolei podczas przeszukiwania grafu pod kątem liczby przesiadek czas wykonania był dłuższy, wynikający z tego, że wiele wierzchołków miało ten sam priorytet zerowej liczby przesiadek.

Algorytm Tabu Search skutecznie znajduje optymalną kolejność zwiedzania miast, a wbudowany mechanizm Cache osiągnął skuteczność (zazwyczaj powyżej 50%) pominiętych zapytań do grafu. Zredukował on tym samym czas obliczeń dla krótkich list z kilkudziesięciu do raptem kilku sekund.

Problemy i trudności

- **Nietypowa reprezentacja czasu w formacie GTFS:** Największym wyzwaniem w tym zadaniu była poprawna obsługa czasu i przesiadek nocnych. W specyfikacji GTFS planowane czasy przyjazdów i odjazdów mogą przekraczać barierę 24 godzin (np. 25:10:00 dla kursów realizowanych po północy). Tradycyjne podejście oparte na formacie godzinowym (HH:MM:SS) i "resetowaniu" doby powodowało gubienie nocnych pociągów oraz błędy przy sprawdzaniu aktywności kursu w kalendarzu. Musiałem przeprojektować graf tak, aby operował na "czasie absolutnym" (całkowitej liczbie sekund od północy dnia początkowego) oraz dynamicznie wyliczał przesunięcia dni, co zagwarantowało ciągłość osi czasu nawet przy wielodniowych podróżach w problemie komiwożera.

Podsumowanie

Zrealizowany projekt to w pełni działający system nawigacji, przystosowany do pracy na rzeczywistych, zależnych od czasu rozkładach jazdy. Podstawą wyszukiwania tras są zoptymalizowane algorytmy Dijkstry i A*, które bez problemu radzą sobie ze śledzeniem upływu czasu i obsługą kalendarza przesiadek. Największym wyzwaniem obliczeniowym był wariant problemu komiwożera, jednak zastosowanie metaheurystyki Tabu Search pozwoliło skutecznie poradzić sobie z jego złożonością. Kluczowe okazały się wprowadzone optymalizacje – w szczególności mechanizm pamięci podręcznej (cache) oparty na "oknach" czasowych. Dzięki niemu udało się drastycznie zredukować liczbę zapytań do grafu, a skrypt potrafi wyznaczyć optymalny plan wieloetapowej podróży w bardzo zadowalającym czasie, sprawnie poruszając się po skomplikowanej siatce połączeń kolejowych.